Python for operations engineering

Packaging and virtual environments, CLI tools, API clients with retries, concurrency, testing with pytest and safe cloud and Kubernetes SDK patterns.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Project structure, packaging and virtual environments (12 min)
2. CLI tools with argparse, click and typer (12 min)
3. API clients: retries, timeouts and pagination (13 min)
4. Concurrency: threads, asyncio and processes (13 min)
5. Testing with pytest and mocking infrastructure (13 min)
6. Cloud and Kubernetes SDKs with safe patterns (13 min)
Cheat sheet

Lesson 1. Project structure, packaging and virtual environments

12 minutes. Lay out an operations tool as a package with pyproject.toml, isolated virtual environments, pinned dependencies and a reproducible install.

A script that works only on your laptop is a liability. A small package with an isolated environment and pinned dependencies installs the same way everywhere.

What you will learn

- A src layout and pyproject.toml
- Virtual environments and why they matter
- Pinning and locking dependencies
- Entry points that become commands
- Reproducible installs in CI and containers

Layout

Put code in `src/mytool/` with tests in `tests/` and metadata in `pyproject.toml`. The src layout stops tests from importing the working directory by accident, so they test what is installed. Keep a short README that says how to install and run.

Virtual environments

A virtual environment gives each project its own interpreter and packages, so one tool upgrading a library does not break another. Create it with `python -m venv .venv`, activate it and install into it. Never install project dependencies into the system Python on a server.

Pinning

A range such as `requests>=2` lets a new release change behaviour overnight. Libraries declare ranges, applications lock exact versions with a tool such as pip-tools, uv or Poetry, and commit the lock file. Update on purpose, run the tests, and read the changelog.

Entry points

Declare `[project.scripts] mytool = "mytool.cli:main"` and the installer creates a `mytool` command. Operators then run a named command instead of `python path/to/script.py`, and it works from any directory.

Reproducible installs

In CI and container builds install from the lock file with hashes where possible, use a slim base image, and copy the lock file before the source so the dependency layer is cached. Record the Python version and fail the build if it is different.

Diagram: Project structure, packaging and virtual environments

Part	What it does	What goes wrong
Source package	src/mytool.	Imports from cwd.
Virtual environment	Isolated packages.	System Python.
Lock file	Exact versions.	Unpinned.
CI and image	Installs from lock.	Drift.

Flow: Source package to Virtual environment / Virtual environment to Lock file / Lock file to CI and image

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A tool that broke after a library release

Incident: *A scheduled job failed because a dependency released a breaking version overnight.*

```
$ pip list --format=freeze | grep -i requests
requests==3.0.0
$ python -c "print('job failed: unexpected keyword argument')"
(this is a simulated illustration, not a real incident)
```

The requirements file used a loose range, so the next build picked up a new major release. The team locked exact versions, committed the lock file and used a scheduled update job that runs the tests before any bump is merged.

Common mistakes

- Installing into the system Python
- Loose version ranges in applications
- Not committing the lock file
- Scripts that depend on the current directory
- No pinned Python version

Check yourself

Why use a virtual environment?

Answer: To isolate project dependencies

What does a lock file do?

Answer: Records exact versions for repeatable installs

What does a console script entry point give?

Answer: A named command from a function

Why the src layout?

Answer: Tests import the installed package, not the folder

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. CLI tools with argparse, click and typer

12 minutes. Build operations tools with clear arguments, exit codes, dry run modes and machine readable output, using argparse from the standard library.

Operators run your tool at bad moments. Clear arguments, safe defaults, a dry run and honest exit codes make it trustworthy.

What you will learn

- Arguments, subcommands and help
- Exit codes and error messages
- Dry run and confirmation for dangerous actions
- Machine readable output
- Choosing argparse, click or typer

Arguments

A good tool has subcommands such as `mytool backup run` and `mytool backup list`, options with defaults and help text for each. `argparse` in the standard library handles this without dependencies, which suits tools installed on servers.

Exit codes

Return 0 on success and a non zero code on failure so scripts and schedulers can react. Print errors to standard error, and make messages say what failed and what to do next. Do not print a stack trace for an expected problem such as a missing file.

Dry run

Destructive tools need a `--dry-run` that prints what would change, and often a required `--yes` or a typed confirmation for production. Default to the safe option, and show the target environment in the output so mistakes are visible.

Output

Provide `--output json` for scripts alongside readable text for people. Write results to standard output and logs to standard error, so piping into other tools works. Keep field names stable, because other tools will depend on them.

Libraries

`argparse` is enough for most tools. `click` and `typer` give less boilerplate, nicer help and testing helpers, at the cost of a dependency. Pick one per team and wrap your logic in plain functions so the CLI layer stays thin and easy to test.

Diagram: CLI tools with argparse, click and typer

Part	What it does	What goes wrong
Parse arguments	Validate input.	Unclear help.
Plan and dry run	Show changes.	Skipped.
Apply	Do the work.	Wrong target.
Exit code and output	Report result.	Always zero.

Flow: Parse arguments to Plan and dry run / Plan and dry run to Apply / Apply to Exit code and output

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: A cleanup tool that ran against production

Incident: *An engineer meant to test a cleanup in staging, but the default target was production.*

```
$ mytool cleanup --older-than 30d
target: production
deleted 1,204 objects
$ echo $?
0
(this is a simulated illustration, not a real incident)
```

The tool defaulted to production and had no dry run, so the first run was real. The team made the environment a required argument, added --dry-run as the default behaviour for destructive commands and printed the target in the first line of output.

Common mistakes

- Defaulting to production
- Always returning exit code zero
- Mixing logs and results on standard output
- Giant functions inside the CLI layer
- No help text

Check yourself

What should a failing tool return?

Answer: A non zero exit code

What does a dry run do?

Answer: Shows what would change without changing it

Where should results and logs go?

Answer: Results to stdout, logs to stderr

Why keep logic outside the CLI layer?

Answer: It is easier to test

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. API clients: retries, timeouts and pagination

13 minutes. Call HTTP APIs safely with timeouts, retries with backoff and jitter, idempotency, rate limit handling and complete pagination.

Most operations scripts are API clients. Without timeouts and sensible retries they hang, hammer a struggling service or silently miss pages of results.

What you will learn

- Always set timeouts
- Retry with exponential backoff and jitter
- Idempotency and which calls are safe to retry
- Rate limits and the Retry-After header
- Pagination without losing items

Timeouts

A request with no timeout can wait forever and hang a scheduled job. Set a connect timeout and a read timeout, for example `timeout=(3, 15)`. Pick values from the service behaviour, and treat a timeout as a failure to handle, not a surprise.

Backoff and jitter

When a call fails with a transient error, wait and try again, doubling the wait each time up to a limit and adding random jitter so many clients do not retry in step. Cap the number of attempts and the total time, then fail with a clear error.

Idempotency

A `GET` is safe to retry. A `POST` that creates something may create it twice. Use idempotency keys when the API supports them, or check for the result before retrying. Retry only errors that can be transient, such as connection errors, 429 and 5xx, not 400 or 404.

Rate limits

APIs return 429 and often a `Retry-After` header. Obey it. Slow down proactively by limiting concurrency, and cache results that do not change. A script that ignores limits may get its credentials blocked.

Pagination

Large lists arrive in pages. Follow the next page token or link until it is absent, rather than assuming one page is all. Watch for items changing during the walk and for page size limits. Write a generator that yields items so callers do not care about pages.

Diagram: API clients: retries, timeouts and pagination

Part	What it does	What goes wrong
API call	Has timeout.	No timeout.
Retry policy	Backoff, jitter.	Retry storm.
Pagination loop	Follows next token.	Stops at page one.
Complete result	All items.	Missing items.

Flow: API call to Retry policy / Retry policy to Pagination loop / Pagination loop to Complete result

Incident: A report that missed most of the data

Incident: *A usage report counted only the first hundred resources because pagination was ignored.*

MARGA by EknathaLabs marga.eknathalabs.com

```
$ python report.py --count
resources: 100
$ python -c "print('next_page_token present: True')"
(this is a simulated illustration, not a real incident)
```

The client read one page and ignored the next page token, so the report looked plausible but was wrong. The team replaced it with a generator that follows tokens until none remain and added a test with a fake API that returns three pages.

Common mistakes

- No timeout on requests
- Retrying non idempotent calls blindly
- Retrying immediately without backoff
- Ignoring Retry-After
- Reading only the first page

Check yourself

Why add jitter to backoff?

Answer: So clients do not retry at the same moment

Which call is naturally safe to retry?

Answer: GET

What should a 429 response trigger?

Answer: Waiting as the Retry-After header says

How do you get all items?

Answer: Follow the next token until it is absent

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Concurrency: threads, asyncio and processes

13 minutes. Choose between threads, asyncio and processes for operations work, limit concurrency with a semaphore and avoid shared state mistakes.

Checking a thousand hosts one by one takes forever. Concurrency helps, but the right kind depends on whether the work waits on the network or burns CPU.

What you will learn

- I/O bound versus CPU bound
- Threads for simple parallel I/O
- asyncio and the event loop
- Processes for CPU work
- Limits, errors and cancellation

Two kinds of work

I/O bound work waits for networks, disks and APIs, so many tasks can overlap. CPU bound work uses the processor, such as compressing or parsing large data. The global interpreter lock lets only one thread run Python code at a time, so threads speed up I/O but not CPU work.

Threads

A `ThreadPoolExecutor` runs a function over many inputs with a fixed number of workers and is the simplest way to parallelise blocking calls such as SSH or HTTP. Keep functions independent, return results rather than mutating shared lists and set a worker limit.

asyncio

`asyncio` runs many tasks on one thread by switching at `await` points. It scales to thousands of connections but needs async libraries, and one blocking call stalls everything. Use `asyncio.gather` to run tasks together and `asyncio.run` to start.

Processes

`ProcessPoolExecutor` uses separate processes and so separate interpreters, which gives real parallelism for CPU bound work. Arguments and results must be picklable, and starting processes has a cost, so use them for heavy tasks.

Limits and errors

Unlimited concurrency overloads the target and your own file descriptors. Use a semaphore or the pool size to limit it. Collect exceptions per task so one failure does not hide the others, and set timeouts so a stuck task does not block the run.

Diagram: Concurrency: threads, asyncio and processes

Part	What it does	What goes wrong
Work items	Many hosts.	Unbounded.
Pool or event loop	Limited workers.	Too many.
Tasks	I/O or CPU.	Wrong model.
Collected results	Per task errors.	Lost errors.

Flow: Work items to Pool or event loop / Pool or event loop to Tasks / Tasks to Collected results

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A health check that opened too many connections

Incident: *A script checking 3000 hosts at once exhausted file descriptors and failed halfway.*

```
$ ulimit -n
1024
$ python check_all.py
OSError: [Errno 24] Too many open files
(this is a simulated illustration, not a real incident)
```

The script started every check at once. The team added a semaphore of 50 concurrent checks, set a per host timeout and collected failures into a report instead of stopping at the first exception.

Common mistakes

- Threads for CPU heavy work
- Blocking calls inside async code
- Unlimited concurrency
- Shared mutable state between threads
- No timeouts per task

Check yourself

When do threads help?

Answer: For I/O bound work

What helps CPU bound work?

Answer: Multiple processes

What does a semaphore do here?

Answer: Limits concurrent tasks

What breaks an event loop?

Answer: A blocking call

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Testing with pytest and mocking infrastructure

13 minutes. Test operations code with pytest, fixtures, parametrisation and fakes for APIs and clouds, so tests are fast and never touch production.

Operations code changes real systems, so it deserves tests more than most. Good tests run in seconds, use fakes and make it safe to change the tool.

What you will learn

- pytest basics and fixtures
- Parametrised tests for edge cases
- Fakes and mocks for external systems
- Testing time, files and the environment
- What to test and what not to

pytest basics

A test is a function whose name starts with `test_` and uses plain `assert`. pytest shows a clear diff when it fails. A fixture is a function that prepares something a test needs, such as a temporary directory, and is passed in by name.

Parametrisation

`@pytest.mark.parametrize` runs one test with many inputs. It suits parsers, validators and size or duration conversions, where edge cases such as empty values, units and negative numbers matter. Name cases so failures read well.

Fakes and mocks

Separate logic from I/O. Pass a client into functions so a test can give a fake one that returns canned responses. `unittest.mock` and the `monkeypatch` fixture replace calls when needed. Over mocking tests the mock instead of the code, so prefer simple fakes with real behaviour.

Time, files and environment

Use `tmp_path` for files, `monkeypatch.setenv` for environment variables and pass the current time in as an argument or use a clock function you can replace. Tests that depend on the real clock or on a developer machine become flaky.

What to test

Test decisions: which resources are selected for deletion, how retries behave, how errors are reported. Add a few integration tests against a disposable environment. Make sure no test can reach production, for example by clearing credentials in the test environment.

Diagram: Testing with pytest and mocking infrastructure

Part	What it does	What goes wrong
Logic functions	Pure where possible.	Mixed with I/O.
Fake client	Canned responses.	Too many mocks.
pytest	Fast tests.	Slow or flaky.
CI gate	Runs on each change.	Skipped.

Flow: Logic functions to Fake client / Fake client to pytest / pytest to CI gate

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: A cleanup that deleted the wrong resources

Incident: *A change to a selection rule removed resources that were still in use, and no test covered it.*

```
$ pytest -q
.....
5 passed
$ python -c "print('no test covers resources tagged keep=true')"
(this is a simulated illustration, not a real incident)
```

The rule for tagged resources had no test, so the change passed. The team wrote parametrised cases for tags, ages and missing fields, ran them with a fake client in CI and required them to pass before any change to the selection logic could be merged.

Common mistakes

- Logic mixed into I/O code
- Mocks that copy the implementation
- Tests that read the real clock
- Tests that can reach production
- No tests for deletion logic

Check yourself

What is a fixture?

Answer: Setup that a test receives by name

What does parametrize do?

Answer: Runs one test with many inputs

How do you make code testable?

Answer: Pass dependencies in and separate logic from I/O

What must tests never do?

Answer: Reach production systems

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Cloud and Kubernetes SDKs with safe patterns

13 minutes. Use cloud and Kubernetes SDKs from Python safely: credentials, pagination, dry runs, tagging, least privilege and audit friendly automation.

SDKs make it easy to change a lot of infrastructure with a few lines. Safe patterns keep automation from becoming the largest risk in the estate.

What you will learn

- Credentials and least privilege
- Listing, filtering and dry run before acting
- Tagging, ownership and audit trails
- Kubernetes client patterns
- Guardrails: limits, approvals and kill switches

Credentials

Automation should use short lived credentials from a role or workload identity, not long lived keys in files. Give it only the permissions it needs, for example read access for a report and delete access only for one tagged set. Separate roles for read and write tools.

List, filter, then act

Build the list of targets first, print it and apply filters such as tags and age. Run in dry run mode by default. Only then act, one item at a time with logging, and stop if the count is much larger than expected.

Tags and audit

Tag everything automation creates with the owner and the tool. Log each action with the account, the resource and the reason, to a place others can read. When something is deleted unexpectedly, the log answers who and why.

Kubernetes patterns

The official client loads configuration from a kubeconfig or from the service account inside a pod. Use label selectors, handle API errors and watch for changes with a timeout and a resource version rather than polling in a tight loop. Use server side dry run where available.

Guardrails

Cap how many resources one run may change, require an explicit flag or approval for large changes, check that the account and region are the expected ones, and provide a kill switch such as a feature flag file. Alert when automation changes more than normal.

Diagram: Cloud and Kubernetes SDKs with safe patterns

Part	What it does	What goes wrong
Scoped role	Least privilege.	Admin keys.
List and filter	Tags and age.	Wrong filter.
Act with limits	Cap and log.	Mass deletion.
Audit log	Who and why.	No record.

Flow: Scoped role to List and filter / List and filter to Act with limits / Act with limits to Audit log

Incident: An automation that tried to delete half the account

Incident: *A cleanup script matched far more resources than intended because a tag filter was empty.*

MARGA by EknathaLabs
marga.eknathalabs.com

```
$ python cleanup.py --dry-run
matched: 4,812 resources
limit per run: 50
aborting: matched count exceeds the limit
(this is a simulated illustration, not a real incident)
```

An empty tag value matched everything. The run stopped because of the per run limit that the team had added after an earlier near miss. They fixed the filter, added a check that rejects empty values and kept the limit and the dry run default.

Common mistakes

- Long lived keys in scripts
- Acting before listing and reviewing
- No cap on changes per run
- No tags or audit logs
- Using admin permissions for convenience

Check yourself

What credentials suit automation?

Answer: Short lived role based credentials

What should a tool do before acting?

Answer: List targets and apply a dry run

Why cap changes per run?

Answer: To limit the damage from a wrong filter

Why tag created resources?

Answer: So owners and tools are traceable

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Python for operations

Packaging, retry rules and pytest commands.

Packaging

Item	Value
Layout	`src/tool/`, `tests/`, `pyproject.toml`
Environment	`python -m venv .venv`
Entry point	`[project.scripts] tool = "tool.cli:main"`

Retry rules

Case	Action
429	Wait for Retry-After
5xx or timeout	Backoff with jitter
4xx other	Do not retry
POST	Use an idempotency key

Pytest

Job	Command
Run	`pytest -q`
One test	`pytest -k name`
Parametrise	`@pytest.mark.parametrize`
Temp files	`tmp_path` fixture

MARGA by EknathaLabs
marga.eknathalabs.com